Lecture 2: Using Haskell

1

Introduction to Functional Programming

Haskell

- More at `http://www.haskell.org/`
- Named after Haskell B. Curry, one of the pioneers of lambda calculus
- First defined in 1990
- We will be using the **Glasgow Haskell Compiler interactive** GHCi (2010) interface, which is
  - freely available
  - Efficient and compact
  - Has a flexible user interface
- Haskell scripts use the **.hs** extension

➢ **Haskell is a general purpose, purely functional programming language**

3

Introduction to Functional Programming

Haskell

- Haskell is a widely used purely functional language with the following features:

❖ **Functional Language**

- In conventional programing language, we instruct the compiler with a series of tasks telling the computer "what to do" and "how to do?"
- But in Haskell we will tell our computer "**what is it**?"

❖ **Modularity**

- A Haskell application is nothing but a series of **functions**
- We can say that a Haskell application is a collection of numerous small Haskell applications

4

Introduction to Functional Programming

Haskell

❖ **Laziness**

- Haskell is a lazy language meaning that Haskell won't evaluate any expression without any reason
- When the evaluation engine finds that an expression needs to be evaluated, then it creates a **thunk data structure** to collect all the required information for that specific evaluation and a pointer to that **thunk data structure**
- A thunk is a subroutine that wraps another function or expression to delay computation until the results are needed
- The evaluation engine will start working only when it is required to evaluate that specific expression

5

Introduction to Functional Programming

Haskell

❖ **Statically Typed**

- In conventional programing language, we need to define a series of variables along with their type
- In contrast, Haskell is a **type interference language**
- By the term, type interference language, we mean the Haskell compiler is intelligent enough to figure out the type of the variable declared
- This means that we need not explicitly mention the type of the variable used

❖ **Maintainability**

- Haskell applications are modular and hence, it is very easy and cost-effective to maintain them

6

Introduction to Haskell

GHC

❖ **Glasgow Haskell Compiler**

- GHC is a state-of-the-art, open-source compiler and interactive environment for the functional language Haskell
- It started as part of an academic research project funded by the UK government at the beginning of the 1990s
- Over its lifetime GHC has generally had around two or three active developers, although the number of people who have contributed some code to GHC is in the hundreds

7

Introduction to Haskell

GHC

❖ **Glasgow Haskell Compiler**

- GHC can be divided into three:
  - The **compilation manager –** which is responsible for the compilation of multiple Haskell source files
  - The **Haskell compiler –** it is abbreviated as Hsc inside GHC, which handles the compilation of a single Haskell source file
  - The **pipeline –** responsible for composing together any necessary external programs with Hsc to compile a Haskell source file to object code

8

Introduction to Functional Programming

GHCi

❖ **Glasgow Haskell Compiler interactive (GHCi)**

- GHCi is an interactive, executable of the GHC **interpreter**
- **Glasgow Haskell Compiler (GHC)** on the other hand, is the de-facto standard Haskell batch compiler
- GHCi is therefore GHC's interactive environment that includes an interactive debugger
- GHCi can
  - interactively evaluate Haskell expressions
  - interpret Haskell programs
  - load GHC-compiled modules

9

Introduction to Functional Programming

Starting Haskell

- Open your terminal and type in `ghci`
- You will be greeted with something like this.

```
GHCi, version 8.10.7: http://www.haskell.org/ghc/  :? for help
Loading package base ... linking ... done.
Prelude>
```

- The **prompt** here is `Prelude>`
- The prompt shows all the modules that have loaded into the interactive session and the Haskell system is waiting for your input
- In this instance, **Prelude** is the module that has been successfully loaded
- Prelude is a module that contains a small set of standard definitions and is included automatically into all Haskell modules
- If several modules have been loaded, the prompt can be `Prelude Data.List Control.Monad>`

11

Introduction to Functional Programming

Starting Haskell

- Prelude is the standard library that is loaded when Haskell is started
- Standard Prelude from `Prelude.hs` extension
- It gives the function signatures for several tasks including
  - Comparison (see the polymorphic equality test operation)
  - Basic I/O :-
    - Simple input
    - Simple output
- Imported by default into other modules

12

Introduction to Functional Programming

Starting Haskell

- If you want to change the prompt, use the `:set prompt` command
- For example `:set prompt "ghci> "` will change the command to `ghci>`

13

Introduction to Functional Programming

Starting Haskell

- If you want to change the prompt, use the `:set prompt` command
- For example `:set prompt "ghci> "` will change the command to `ghci>`

14

Introduction to Haskell

GHCi Commands

- GHCi commands all begin with `:`
- The `:` is then followed by a single command name and then zero or more parameters
- The command name may be abbreviated, with ambiguities being resolved in favour of the more commonly used commands
- Examples include:
  - `:help` or `:?` – displays a list of the available commands
  - `:` - repeats the previous command
  - `:quit` - quits GHCi, this can also be achieved by using `ctrl + D`
  - `cd dir` – changes current directory to dir
  - `:set prompt prompt` – sets the string to be used as the prompt in GHCi

15

Introduction to Haskell

Evaluating Expressions

- An **expression** applies a function to numbers and to other functions
- Consider the mathematical expression $1 + 2 \times 3$
- To evaluate the expression, we use mathematical rules to work it out, and stop when the expression can not be processed any further
- We might show the process like this:

  $1 + \underline{2 \times 3}$

  $\Rightarrow \underline{1 + 6}$

  $\Rightarrow 7$

- Expressions in Haskell are evaluated the same way using mathematical operators and operator precedence

16

Introduction to Haskell

Evaluating Expressions

- FP evaluates expressions to give values
- Haskell as a FP language is like an electronic calculator, it can evaluate expressions to give values
- Haskell will evaluate expressions typed at the prompt
- GHCi interprets the whole line as an expression to evaluate
- The expression may not span several lines - as soon as you press enter, GHCi will attempt to evaluate it



17

Introduction to Haskell

Evaluating Expressions

- **Negative numbers** are written with a – before them
- If they are used after an operator, enclose negative numbers in brackets



18

Introduction to Haskell

Evaluating Expressions

- Boolean operators are used for comparisons, with only two values, True and False
- Values can be compared to each other

```
GHCi, version 8.10.7: https://www.haskell.org/ghc/  :? for help
Prelude> 20 < 13
False
Prelude> 20 >13
True
Prelude> 2 == 4
False
Prelude>
```

- Boolean values are combined using `&&` for and and `||` for or
- The && operator has higher precedence than the || operator
- Every value and every expression has a **type** (later)

➢ **An expression always has the same type as the value it will evaluate to**

19

Introduction to Haskell

Evaluating Expressions

- The comparison operators are shown below

a /= b	True if a is not equal to b
a == b	True if a and b are equal
a < b	True if a is less than b
a > b	True if a is greater than b
a <= b	True if a is less than or equal to b
a >= b	True if a is greater than or equal to b

20

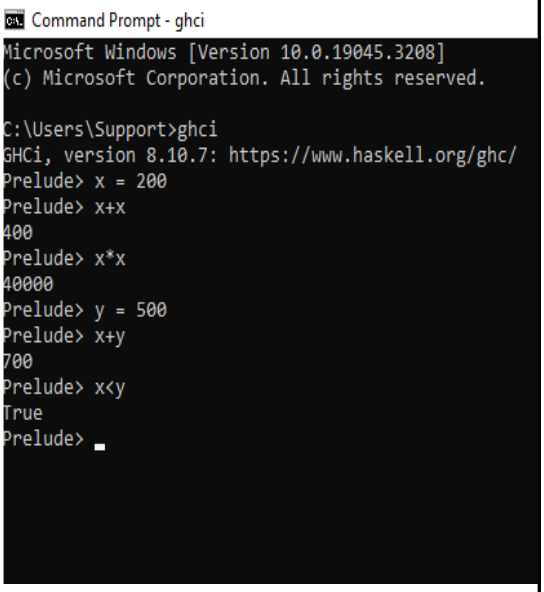
Introduction to Haskell

Evaluating Expressions

- Users are also allowed to define an expression to be evaluated by typing it at the prompt

➢ The names can be used to refer to the expressions at a later stage

- The keyword `let` is used before an expression to define it
- **Expressions evaluated at the prompt in this manner are lost when GHCi is closed**



21

Introduction to Haskell

Evaluating Expressions

- Using the keyword `let` to define expressions

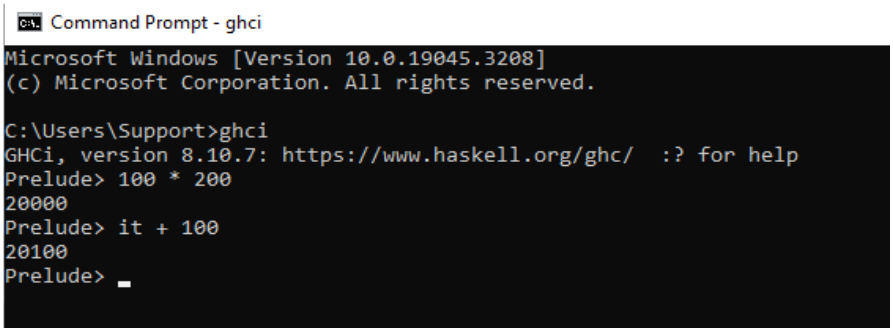
```
GHCi, version 8.10.7: https://www.haskell.org/ghc/  :? for help
Prelude> let cube x = x * x * x
Prelude> cube 1
1
Prelude> cube 2
8
Prelude> cube 3
27
Prelude> cube 15
3375
Prelude> cube 3000
27000000000
Prelude>
```

23

Introduction to Haskell

Evaluating Expressions

- The keyword `it` can be used to refer to the value from the most recently evaluated expression
- This is useful if an expression had not been named prior to use



24

Introduction to Haskell

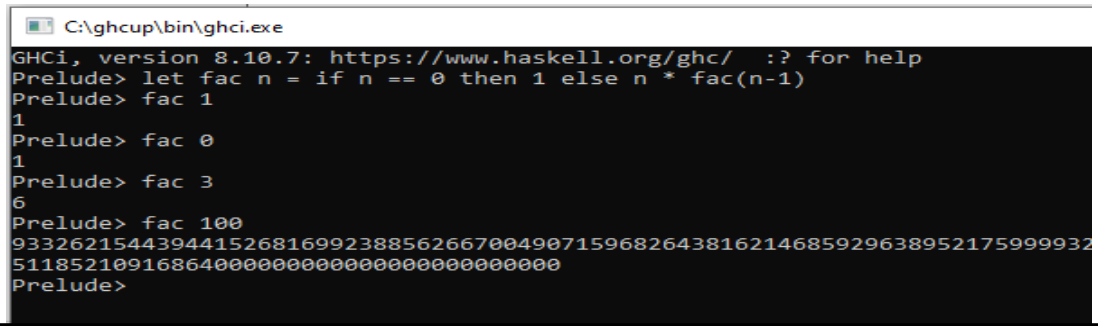
Evaluating Expressions

❖ **Example**

- Le us define a function called **fac** which computes the factorial of an integer

```
Prelude> let fac n = if n == 0 then 1 else n * fac (n-1)
```

- We can now run **fac** on some argument



25

Introduction to Haskell

Evaluating Expressions

❖ **If statements**

- If statements are a bit different in Haskell than they are in other languages
- For instance, in Java, an if statement can exist without an else

```
if (a <= 2) {
  a = 4;
}
```

26

Introduction to Haskell

Evaluating Expressions

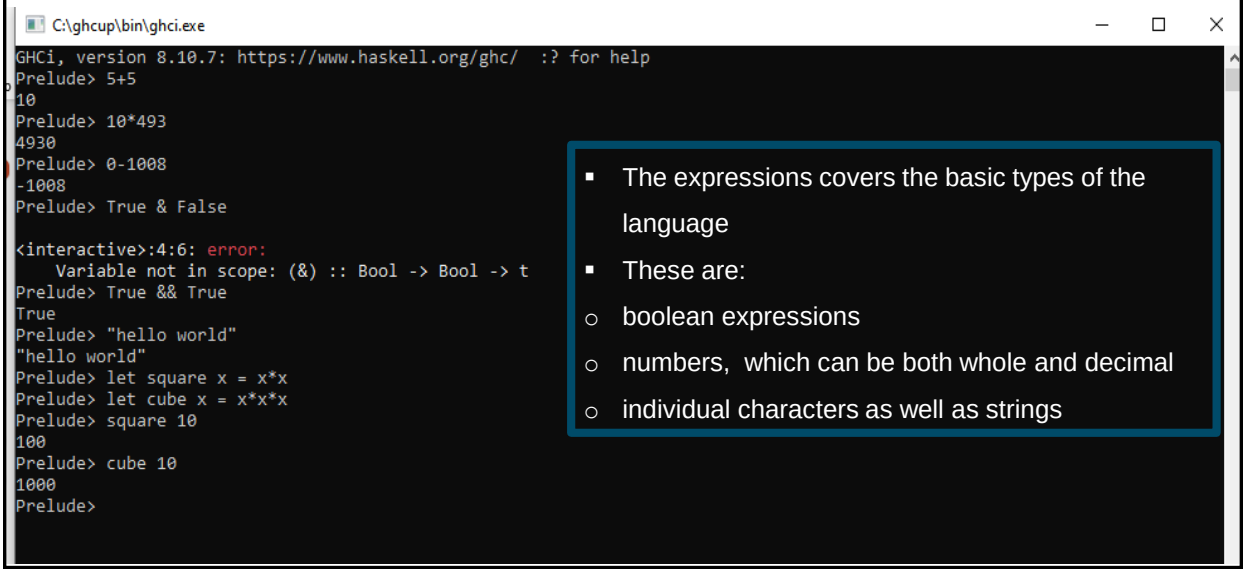
- In Haskell, all if-statements **must have an else branch**!
- This is because everything in Haskell is an expression, and every expression has a type
- For example, lets create a function **isNegative** that determines whether a number is negative or not

```
Prelude> isNegative a = if a < 0 then True else False
Prelude> isNegative 1
False
Prelude> isNegative (-5)
True
Prelude>
```

27

Introduction to Haskell

Evaluating Expressions



28

Introduction to Haskell

Evaluating Expressions

- Some **expressions can be too long when typed at the prompt in one line**
- For these expressions, Haskell allows expressions or functions to be typed over several multiple lines
- The entire expression with is enclosed in brackets `:{` and following it with `:}`
- Every line is terminated by pressing the Enter key as usual
- Haskell knows we reached the end when we type `:}` followed by the Enter key
- This allows long functions or programs to also be created and evaluated at the prompt
- Haskell expects the second line to start a little to the right of the function name, i.e. indented of the name of the function
- **Indentation** is important, so ensure that the lines are indented properly by using the space bar

29

Introduction to Haskell

Evaluating Expressions

❖ **Example**:

- A function called `isVowel` that determines if a letter is a vowel or not

Start of the long expression is indicated by :{

Several lines of the same expression, indicated by hitting the Enter key

The name of the function should always be to the left : **Indentation**

End of the long expression indicated by :}

```
GHCi, version 8.10.7: https://www.haskell.org/ghc/  :? for help
Prelude> :{
Prelude| isVowel x =
Prelude|     x == 'a' || x =='e' || x == 'i'
Prelude|     || x == 'o' || x == 'u'
Prelude| :}
Prelude> isVowel a

<interactive>:6:9: error: Variable not in scope: a :: Char
Prelude> isVowel 'a'
True
Prelude> isVowel 'f'
False
Prelude> isVowel p

<interactive>:9:9: error: Variable not in scope: p :: Char
Prelude> isVowel 'p'
False
Prelude> isVowel 'e'
True
Prelude>
```

Error!!

30

Introduction to Haskell

Evaluating Expressions

❖ **Example:**

- A function `myIfStatement`

C:\ghcup\bin\ghci.exe

```
GHCi, version 8.10.7: https://www.haskell.org/ghc/  :? for help
Prelude> :{
Prelude| myIfStatement a = if a <=10
Prelude|     then a + 10
Prelude|     else a - 10
Prelude| :}
Prelude> myIfStatement 1
11
Prelude> myIfStatement 10
20
Prelude> myIfStatement 15
5
Prelude> myIfStatement 100
90
Prelude>
```

31

Introduction to Haskell

The Hello World Program

- A Haskell program can easily be run interactively by typing at the prompt
- For example, the famous hello world program can easily be created as:



- Haskell evaluates the string and the result printed
- Another way is to print directly to standard output

```
Prelude> putStrLn "Hello World"
Hello World
```

How easy is that!!!

32

Introduction to Haskell

GHCi Commands

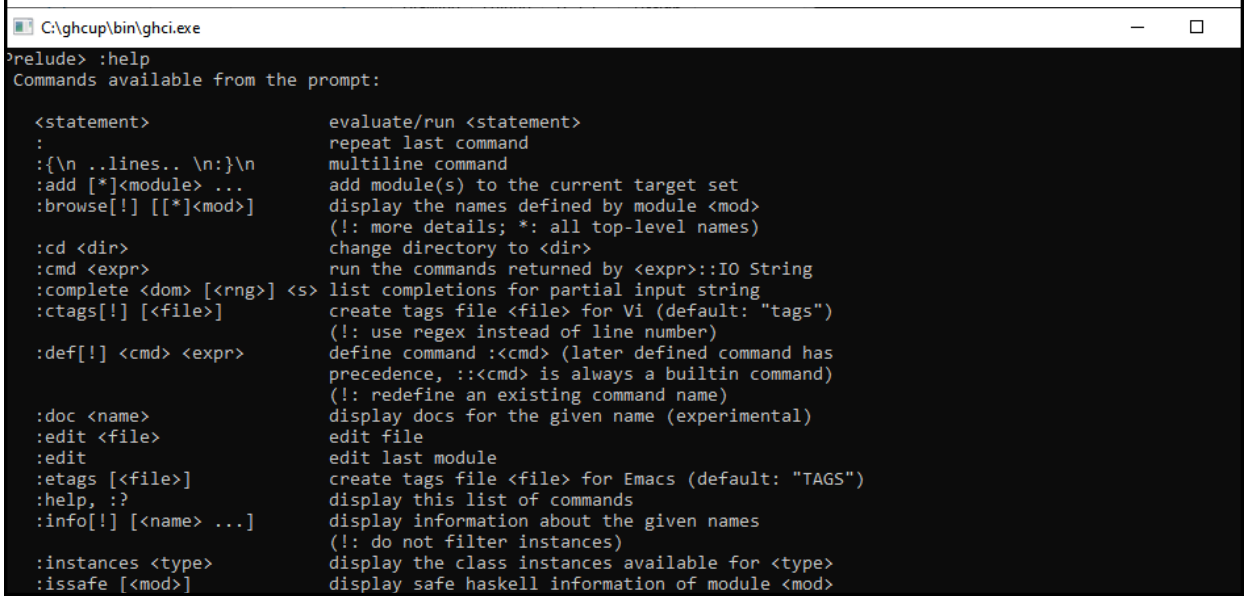
- GHCi commands begin with a :
- Some of the commands can be abbreviated to their first letter only
- Example:
  - To quit GHCi, the command is `:quit`, but it can also be abbreviated to `:q`
- The `:help` command gives a list of all the commands available from the prompt
- Can also use `:h` or `:?`
- To get the type of an expression, use the `:type exp` command, where `exp` is the expression to be evaluated:

```
Prelude> :type cube
cube :: Num a => a -> a
Prelude>
```

33

Introduction to Haskell

GHCi Commands



34

Introduction to Haskell

Haskell Module

- A Haskell program is a collection of **modules**
- The main module loads up the other modules and then uses the functions defined in them to do something
- Having code split up into several modules has several advantages:
  - if a module is generic enough, the functions it exports can be used for many different programming tasks
  - if your own code is separated into self-contained modules which don't rely on each other too much (we also say they are loosely coupled), you can reuse (some of) them separately later on
  - splitting code into several parts, each of which has a specific purpose, makes writing code more manageable

35

Introduction to Haskell

Haskell Module

- The Haskell standard library is split into modules
- Each of them contains functions and types that are somehow related and serve a common purpose
- Module names are alphanumeric and must begin with an uppercase letter
- When creating modules, the declaration should begin with the keyword **`module`**
- Modules can be imported using the keyword **`import`**
- This must be done before defining any functions, so imports are usually done at the top of the file
- We can import several modules in one script, we just have to put each import statement on a separate line

```
import < moduleName1>
import < moduleName2>
import < moduleName3>
. . .
```

36

Introduction to Haskell

Haskell Module

- If we just need a couple of functions from a module, we can selectively import just those functions
- If we want to import only the `nub` and `sort` functions from `Data.List`, we do this:

```
import Data.List (nub,sort)
```

- We can also choose to import all of the functions of a module except a few select ones
- This is often useful when several modules export functions with the same name and we want to avoid name conflicts
- For example, suppose we already have our own function that's called `nub` and we want to import all the functions from `Data.List` except the nub function
- Then we do the following:

```
import Data.List hiding (nub)
```

37

Introduction to Haskell

Haskell Modules

❖ **Examples of Haskell library modules**

- **List** provides some wonderful functions to work with **list** type data with a wide range of functions
- The **Char** module has plenty of predefined functions to work with the Character type
- **Map** is an unsorted value-added pair type data type. It is a widely used module with many useful functions
- The **Set** module has some very useful predefined functions to manipulate mathematical data
- A set is implemented as a binary tree, so all the elements in a set must be unique

38

Introduction to Haskell

Haskell Modules

- To create a custom module, use the keyword `module` and `where`
- The module can then be called at other programs
- To implement this custom module, we will create a separate file called

```
module CustomModule where

myFirstExpression::String

myFirstExpression = "Hello World!"
```

- This creates a module called CustomModule
- We can then save the module in a file called custom.hs

39

Introduction to Haskell

Haskell Programs

- Expressions typed at the prompt are lost when GHCi exits
- Expressions can be evaluated using definitions added to a **program** in a **script**
- In this case they can be used later and can also be used to in other expressions without having to type them

40

Introduction to Haskell

Haskell Script

- Expressions evaluated by typing at the prompt are lost when GHCi is closed
- Therefore, we need to have a way to keep some expressions for use later without having to type them by typing the expressions in a **program**
- A Haskell program is also called a **script**
- Any text editor can be used to create the scripts
- Haskell scripts are stored with a `.hs` extension
- Only these files with a .hs extension will be loaded, and the extension can be omitted when they are loaded with GHCi is launched or by using the `:load` command

41

Introduction to Haskell

Compile Script

- Instead of running interactively, the scripts can be compiled to create an executable files

```
ghc -o hello hello.hs
```

- The -o option lets you specify the name of the output file
- The input file in this example is hello.hs
- We have to define the `main` function has to be defined in the input file
- When the program is run that will be the main entry point, just like all other languages

42

Introduction to Haskell

Compile the Program

❖ **To run the script**

- Ensure you are in the correct directory where your script is saved
- As an example, assume that our custom.hs is saved in C:\CSI243\Labs, the we can compile the script by typing `runhaskell custom.hs` at the prompt
- Remember that we can also leave out the file extesion

```
C:\CSI243\Labs> runhaskell custom.hs
```

- The script is executed and `Hello, world!` printed to the screen

43

Introduction to Haskell

Compile the Program

- Tu compile using GHC

```
C:\CSI243\Labs> ghc custom.hs
 [1 of 1] Compiling CustomModule             ( custom.hs, custom.o )
```

- Two new files are created in the same directory
  - custom.o
  - custom.hi

44

Introduction to Haskell

Comments

- A Haskell program also contains **Comments**
- A comment is a piece of information for the program user rather than the computer
- Typically contain an explanation about the function
  - how it works
  - how it should not be used
- Can be indicated in two ways:
  - `--` this is for a short comment that may share a line with a symbol, placed to the right
  - `{-` this is for comments that a long, spanning more than three lines. The comments can be of any length, even enclosing other comments. i.e., **nested comments** `-}`

46

Introduction to Haskell

Comments

```
-- This is a comment <- this is syntax highlighted as though it were code
{- This is also a Haskell comment -}


 // This isn't even a comment in Haskell, it's a syntax error
 # This isn't a comment either
-- // Only this is a comment


-- Some actual Haskell code:
 f :: a -> a
 f x = x
```

47

Introduction to Haskell

Comments

- **Haskell's nested-comment style** can be used for documentation annotations, which is sometimes more convenient when using multi-line comments
- Everything between {- followed by a space and -} is a **block comment**
- Example:

```
{-
 The 'square' function squares an integer.
 It takes one argument, of type 'Int'.
-}
square :: Int -> Int
square x = x * x
```

48

Introduction to Haskell

The `main` Function

- **The entry point** to any Haskell program is a function called **main**
- Every program needs to have one function called `main`, which is its entry point
- That's where the execution starts and ends
- In Haskell, `main` is special in other ways as well
- For now, it's enough to know that you can do I/O from main and that the syntax for doing I/O is slightly different than for regular functions
- However, if all your I/O is just a single call to print, you can just call it from main

49

Introduction to Haskell

Other Functions

- The basic syntax of a function definition is pretty simple: function name followed by argument names, then an equal sign, and an expression
- For instance, here's the function that squares its argument:

```
-- show This is a function definition
square x = x * x
```

50

Introduction to Haskell

Haskell Commands

- To quit GHCi use the quit command `:quit` or simply use `:q` at the prompt

```
ghci> :quit
Leaving GHCi.
```

- Other useful command include:
  - `:help` – gives a complete list of GHCi commands
  - `:load Square` – load the Haskell module Square.hs, note that the extension is omitted
  - `:info Square` – gives information about the object Square
  - Up or down arrow – move up and down the command history

51

Exercises

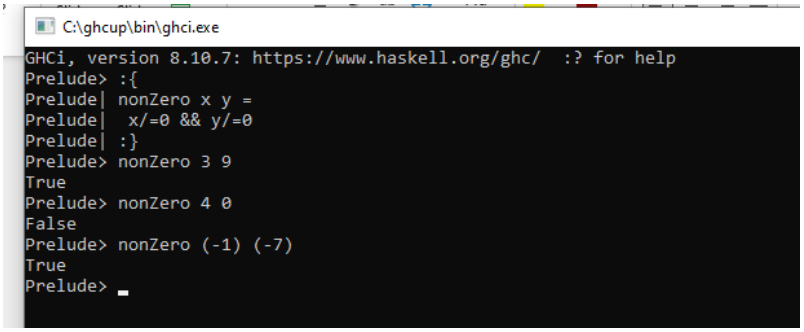
1. A function that multiplies a given number by 10 called `multTen`



53

Exercises

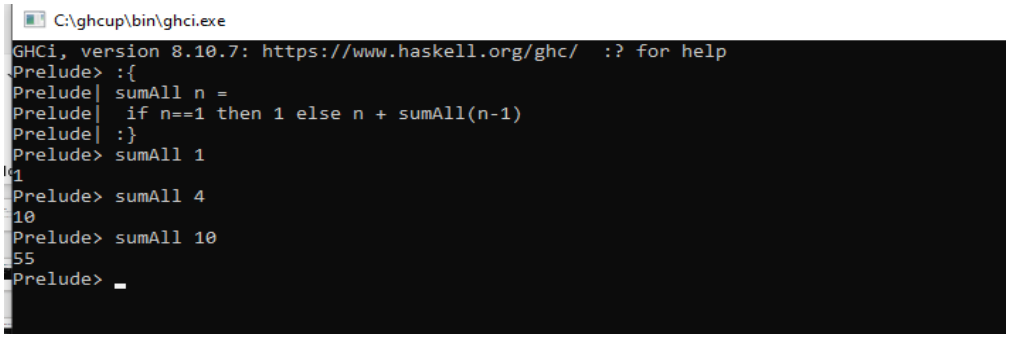
2. A function which returns True if both its arguments are non zero, and False otherwise



54

Exercises

3. A recursive fuction recursiveSum which, when given a number n, calculates the sum of 1 + 2+3..+n



55

Introduction to Haskell

Haskell Commands

- **Modules** allow programs to be broken into to smaller manageable components
- Each module has a name which starts with a capital letter
- A module contains a collection of Haskell definitions
- To define a module

```
module Car where  -- defines a module called Car
.......
```

- The convention used is that of one module per file
- The file name corresponds to the module name e.g the file `Cars.hs` contains the module `Cars`
- A module may import definitions from other modules using the keyword `import`

```
module Car where   -- module is called Car
import Van         -- import the definitions in the Van module
.....
```

58

Introduction to Haskell

Creating a program

1. Create the program using any text editor
2. Save the program using the `.hs` extension as `myFile.hs`
3. Start Haskell GHCi
4. At the prompt, type `:load myFile.hs` which is used to load a Haskell source file into GHCi

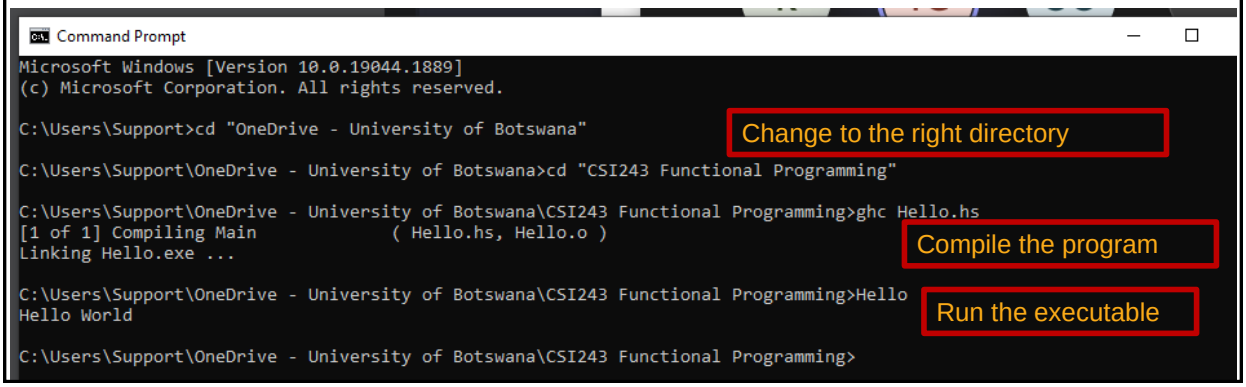
```
Prelude> :load myFile
[1of 1] Compiling Main              ( myFile.hs, interpreted )
Ok, modules loaded: myFile.
*Main>
```

63

Introduction to Haskell

Compiling The program

- **Compile** the program using the GHC compiler
- Ensure that you are in the directory where the program is saved
- This results in an executable file `Hello.exe` that can then be run
- Running the executable simply prints `Hello World` to the screen



64

Introduction to Haskell

Compiling the program

- We can also use the command `ghci Hello.hs` at the prompt to compile the the program
- Then to execute the program, type `main` at the `Main>` prompt
- Remember, to quit GHCi, use the `:quit` or `:q` command

```
C:\Users\Support\OneDrive - University of Botswana\CSI243 Functional Programming>ghci Hello.hs
GHCi, version 8.10.7: https://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main             ( Hello.hs, interpreted )
Ok, one module loaded.
*Main> main
Hello World
*Main> :q
Leaving GHCi.

C:\Users\Support\OneDrive - University of Botswana\CSI243 Functional Programming>
```

65

Introduction to Haskell

Haskell Commands

- If you make some changes to the source code and want GHCi to recompile the program, use the `:reload` command
- The program will be recompiled as necessary, with GHCi doing its best to avoid actually recompiling modules if their external dependencies haven't changed
- This is the same mechanism we use to avoid re-compiling modules in the batch compilation setting

66

Introduction to Haskell

Defining Functions

- The basic syntax of a function definition is pretty simple: function name followed by argument names, then an equal sign, and an expression
- For instance, here's the function that squares its argument: `square x = x * x`
- There are no parenthesis or punctuations between the function name and the arguments
- Example

```
a b c d =  b * c * d
```

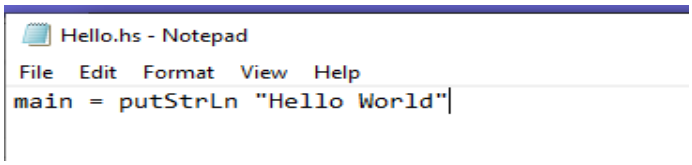
this is a definition of a function a that takes three arguments, b, c, and d

68

Introduction to Haskell

Creating a program

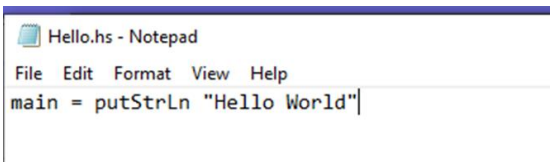
- Create the program `Hello.hs` to display Hello World to the screen, rather than typing at the prompt
- Remember you can easily type the program in a text editor, in this instance we are using Notepad, the extension should be `.hs`



69

Introduction to Haskell

`main` Module

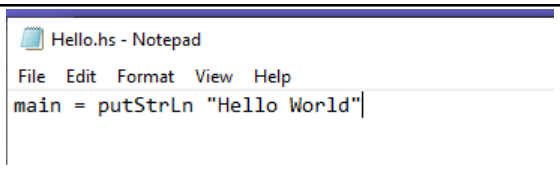


- The `main`, module is the program's entry point
- That's where the execution starts and ends
- You can do I/O from main and that the syntax for doing I/O is slightly different than for regular functions
- However, if all your I/O is just a single call to print, you can just call it from main
- **I/O actions will only be performed** when they are given a name of main or when they're inside a bigger I/O action that we composed with a do block
- We can also use a do block to glue together a few I/O actions and then we can use that I/O action in another do block and so on

70

Introduction to Haskell

putStrLn Fuction



- `putStrLn` is one of the standard Prelude tools
- It is a function that takes a string variable and displays it to the terminal
- The function `putStr` will do the same, but with no print line break
- So, using the = operator, the program binds the name `main` to the results of the function `putStrLn` applied to the string “Hello Word”

➢ We just defined a name called **`main`** and in it we call a function called **`putStrLn`** with the parameter **`"hello, world"`**

72

Introduction to Haskell

putStrLn Fuction

- `putStrLn` function takes a String
- However, it does not get something back that it can use in another function
- Using the :type command, it shows:

```
Prelude> :t putStrLn
putStrLn :: String -> IO ()
```

73

Introduction to Haskell

The putStrLn Fuction

```
Prelude> :t putStrLn
putStrLn :: String -> IO ()
```

- **IO** stands for input and output
- Wherever there is IO in a type, interaction with the world outside the program is involved, called **IO values actions**
- The **()** in IO() , is the type of the return value of the action; that is, the type of what it gives back to the program
- () , pronounced as **unit**, is a type that only contains one value also called (), which is effectively a tuple with zero elements
- Since putStrLn sends output to the world but doesn't return anything to the program, () is used as a placeholder
- We might read IO () as "action which returns ()"

74

Introduction to Haskell

The putStrLn Function

- IO can also be used to:
  - print a string to the screen
  - read a string from a keyboard
  - write data to a file
  - read data from a file

```
Prelude> :t putStrLn
putStrLn :: String -> IO ()
```

- A complete Haskell program is actually a big IO action
- In a compiled program, this action is called `main` and has type IO ()
- From this point of view, to write a Haskell program is to combine actions and functions to form the overall action main that will be executed when the program is run
- The compiler takes care of instructing the computer on how to do this

75

Introduction to Haskell

The putChar Function

- The putChar takes a character and returns an I/O action that will print it out to the terminal

```
main = do
   putChar 't'
   putChar 'h'
   putChar 'e'
```

- When, run, it will print `the` on the screen
- `putStr` is actually defined recursively with the help of putChar

76

Introduction to Haskell

The print Function

- The function `print` calls show function with that value to stringify it and then outputs that string to the terminal
- It first runs `show` on a value and then feeds that to `putStrLn`, which returns an I/O action that will print out our value
- Compared to `putStrLn`, when you print a string type, it is printed with quotes around
- Other types do not have quotes around
- Therefore, to print out strings, use `putStrLn` because we don't want the quotes around them, but for printing out values of other types to the terminal, `print` is used the most
- When we type out a value at the prompt, and press the return key, GHCI actually uses print on that value to display it on our terminal

77

Introduction to Haskell

The do Block

```
main = do
    putStrLn "The answer is: "
    print 43
```

- Haskell has special syntax for sequencing
- It's called the **do notation**
- We do this by putting them inside a `do` block
- The do syntax to glue together several actions into one

```
main = do {
    putStrLn "The answer is: ";
    print 43
}
```

- The block is constructed using line breaks and proper indentation
- In some languages blocks are usually delimited by { } and statements are separated by semicolons
- In fact, you can do the same in Haskell, although such notation is rarely used

79

Introduction to Haskell

The do Block

- As an example, we can create a program that interacts with the user asking them their name then prints Hello Name to the screen
- The program and the results are shown below:

```
main = do
 putStrLn "Hello... What is your name? "
 name <-getLine
 putStrLn ("Hello there " ++ name ++ " !!")
```

```
C:\Users\Support\OneDrive - University of Botswana\CSI243 Functional Programming>ghc HelloName.hs
[1 of 1] Compiling Main             ( HelloName.hs, HelloName.o )
Linking HelloName.exe ...

C:\Users\Support\OneDrive - University of Botswana\CSI243 Functional Programming>HelloName
Hello... What is your name?
Tebogo
Hello there Tebogo !!

C:\Users\Support\OneDrive - University of Botswana\CSI243 Functional Programming>
```

80

Introduction to Haskell

The do Block

- Note that in a *do* block, **the last action cannot be bound to a name**
- This is because the *do* block automatically extracts the value from the last action and binds it to its own result.
- This means that, after any line where you use <- to get the value of an action, Haskell expects another action, so the final action cannot have any <-

81

Introduction to Haskell

The do Block

- Example, removing the last statement from the the last example:

```
main = do
 putStrLn "Hello... What is your name? "
 name <-getLine
```

- This gives an error

```
C:\Users\Support\OneDrive - University of Botswana\CSI243 Functional Programming>ghc docode.hs
[1 of 1] Compiling Main             ( docode.hs, docode.o )

docode.hs:3:2: error:
    The last statement in a 'do' block must be an expression
      name <- getLine
  |
3 |  name <-getLine
  |  ^^^^^^^^^^^^^^

C:\Users\Support\OneDrive - University of Botswana\CSI243 Functional Programming>
```

82

Introduction to Haskell

The getLine Fuction

```
ghci> :t getLine
getLine :: IO String
```

- `getLine` is an I/O action that, when run, will return a result type of String
- It waits for the user to input something at the terminal and then that something will be represented as a string
- The string typed in can then be bound the variable name, which will then have a type of **`String`** as well
- To find the type of the function getLine use `:type` of `:t` command
- getLine doesn't actually take an argument, it takes as input whatever is in the line in the terminal
- This means that the value is not know until it is typed in
- `getLine` is said to be **impure** because its result value is not guaranteed to be the same when performed twice

83

Introduction to Haskell

The getLine Fuction

- To get the value out of an I/O action, you have to perform it inside another I/O action by binding it to a name with **<-**
- Therefore, the statement `name = getLine` will only give the GetLine action the name `name`

➢ It will **not** read from the input and bind the value of that to `name`

84

Introduction to Haskell

The `let` Fuction

- In Haskellthe `let`, function is used for binding variables, which are **local**, that is values inside of a function that only that function can see
- We can define any type of variable using `let` keyword before the variable name in Haskell
- But their scope is local, we also have let in Haskell which is another form of defining the variable and use them after it
- `let` is rather used to binding of the variable values and used to perform desired operations on them
- It is used as the alterative of '`where`', so we can say let in is an expression which is used to define and use variable at very local scope in Haskell
- To use the let function, the syntax is `let variable = somevalue` allows us to bind the value to the variable, which can then be used locally

  e.g. `let x = 100`

86

Introduction to Haskell

The `let` Fuction

- The let x = 100 will only bind one value to the variable
- To define multiple variables at once, use the `let in` expression also provided by Haskell library
- By using let … in … we can define several variables or we can say we can bind multiple variables
- This expression allows us to define the variable anywhere but that will be very local, and we cannot scope them using guard in Haskell programing

87

Introduction to Haskell

The `let` Fuction

- Example

```
(let x = 10 in x + 100) + 200
```

- We are using the let in function to define and calculate it
- First, x is defined with a value of 10
- Then, after this, it is used in part to continue the calculation
- This is where 100 is being added to the existing variable defined in the let part of the expression
- After the complete le in expression, 200 is then added

88

Introduction to Haskell

The read Function

- The read function is used to deal with strings
- It is used if you want to parse any string to any particular type
- Like the putStr and the getLine functions, `read` function is an in-built function, that means we do not require to include or install any dependency for it, we can use it directly
- The `read` function is used for parsing of string values like other programming language, not only single value we can parse the array as well using read function
- To use the read function use `read "string ":: type` will convert string of type `string` to type `type`
- For example `read "1234":: Float will` convert the string 1234 to a floating number 1234

89

Introduction to Haskell

The read Function

- Example
- Using the read function to convert from string to Integer

```
main = do
print("Demo to show usages of read function in haskell !!")
print("calling read function here")
let val1 = read "10" :: Int
let val2 = read "20" :: Int
let val3 = read "30" :: Int
let val4 = read "40" :: Int
let val5 = read "50" :: Int
let val6 = read "60" :: Int
print("printing the result here ..")
print("result one is ::", val1)
print("result two is ::", val2)
print("result three is ::", val3)
print("result four is ::", val4)
print("result five is ::", val5)
print("result six is ::", val6)
let mult = val1*val2*val3*val4*val5*val6
putStr "All multiplied = "
print mult
```

90

Introduction to Haskell

Indentation

- Indentation in programming helps in readability
- For example indentation in the definition of the functions helps in readability
- Haskell is very particular about indentation
- Programs will not be accepted unless they are properly indented

91

Introduction to Haskell

Indentation

```
sumSquare :: Integer -> Integer -> Integer

sumSquare x y
 = sqX + sqY
 where
 sqX = x * x
 sqY = y * y
```

- The definitions must be aligned properly, otherwise there will be an error
- Mixing tabs and spaces will very often result in compiler errors
- If using tabs, ensure that they are always 8 spaces long, or you're likely to run in to problems.
- The where clause must be within the definition it belongs to, to the right of the start of the definitions

92

Introduction to Haskell

Indentation

- The **scope** of a definitions is the part of the program in which the definitions can be used
- All definitions at the top level have the whole scripts they are defined in as their scope , that is they can be used in all definitions the script contains
- They can also be use in definitions that occur before theirs in the script
- Local definitions on the other hand, are not visible in the whole script
- They are just visible in the equation in which they appear

93

Introduction to Haskell

Indentation

- A script can have several definitions one after the other
- In Haskell, the layout is important in showing where one definition ends and the other one starts
- It is advisable to give all definitions the same level of indentation
- Always the start the top-level definitions on the left side

```
multAll a b c d -- remember that names in definitions  begin with small letters
 next line is indented
```

- If you want to write more than one definitions in one line, separate them with a `;`

```
e.g. add = 45; multAll = 100
```

- There are words that are **reserved and can't be used as name:**
- Examples include: `case, class, data, default, do, if, else, import, module`

94

Introduction to Haskell

Case Sensitive

- Haskell is **case-sensitive**
- Haskell distinguishes between values (for instance, numbers: 1, 2, 3, . . .); strings: "abc", "hello", . . . ; characters: 'a', 'b', ' ', . . . ; even functions: types for instance, the function squares a value, or the square-root function); and types (the categories to which values belong)
- Haskell requires that the
  - names given to functions and values begin with a lower-case letter
  - names given to types begin with an upper-case letter
- The moral is: if your otherwise correct program won't compile, be sure you haven't named your function Foo, or something else beginning with a capital letter

95

Examples

```
{-
This is a very simple "program" written in Haskell. It defines a module calledTest.
In this module, there are three definitions: x, y and z.
-}
module Test
  where
  x = 5
  y = (6, "Hello")
  z = x * fst y
```

- There are two predefined functions that allow you to extract the first and second elements of a pair
- They are, respectively, `fst` and `snd`, and they only work for pairs
- The `where` statement follows the name of the module and represents the starting point of the code
- Compile the program typing `ghci Test.hs` at the prompt
- Remember that the module name should be the same as the file name

97

Examples

```
Test> x
5
Test> y
(6,"Hello")
Test> z
30
```

- Notice that the prompt says Test rather than Prelude
- That means that Test is the current module
- Remember that Prelude is always loaded and contains the standard definitions (for instance, the (:) the Prelude operator for lists, or (+) or (*), fst, snd and so on)
- Now that we've loaded Test, we can use things that were defined in it

98

Examples

- A simple program that asks a user for his name and then address him directly

```
module Main
 where
 main = do
 putStrLn "Please enter your name: "
 name <- getLine –run the command getLine and store results in name
 putStrLn ("Hello, " ++ name ++ ", how are you?")
```

- This creates a module called `Main`, which can be compiled
- The primary function is called main, so that the compiler knows to run it when the program is run
- After compiling the code, the prompt is shown as `Main>`

```
Main> main
Please enter your name:
Thato
Hello, Thato, how are you?
Main>
```

99

Exercise

- Write a program which asks the user for the base and height of a right angled triangle, calculates its area, and prints it to the screen
- The interaction should look something like:

```
Enter the base:
3.3
Enter the height?
5.4
The area of that triangle is 8.91
```

- You will need to use the function `read` to convert user strings like "3.3" into numbers like 3.3

103

Exercise

- A program sample is shown below:

```
main = do
 putStrLn "Enter the base: "
 name <-getLine
 let base = read name::Float
 putStrLn "Enter the height: "
 name <-getLine
 let height = read name::Float
 let area = base*height
 putStrLn ("The area of the triangle is :  " )
 print(area)
```

104

Introduction to Haskell

The If Statement

- The if statement which is used to check the specific condition and based on it we execute the logic of our application
- If statement is always followed by the else statement in any programming language
- The if block or statement will always be executed if the expression is evaluating to be true, if it is not true then else part will be executed accordingly
- In short, if statement is said to be a conditional statement, which is used to check the given expression is true or false

```
if (the condition statement) then (true block ) else (false block )
```

- In Haskell, you must have both a then and an else clause in the if then statement

105

Introduction to Haskell

The If Statement

```
main = do
  let demo = 10
  if demo == 10
  then putStrLn "equal"
   else putStrLn "not equal"
```

- As you can see in the above line of code we are trying to compare our variable with some value.
- First, we have defined the main module, and inside that, we are writing our logic with the if signature
- First, we have declared one variable named ‘demo’ which contains a value of 10
- After that, we have defined the if statement, which is comparing the demo variable with some value, if the value is the same then if block will be executed, otherwise it will execute the else statement in the output.

106

Introduction to Haskell

The case Statement

- Haskell, like many other languages, also supports `case` constructions
- These are used when there are multiple values that you want to check against
- The example below defines a function called `caseFunction` that has a value of 1 if its argument were 0; a value of 5 if its argument were 1; a value of 2 if its argument were 2; and a value of −1 in all other instances
- Writing this function using if statements would be long and very unreadable; so we write it using a case statement as follows

```
caseFunction  x =
 case x of
 0 -> 1
 1 -> 5
 2 -> 2
 - -> -1 -- the underscore iss a “wildcard” – it will match anything)
```

- Indentation is important here

107

Introduction to Haskell

The case Statement

- The same code can be written using braces and semicolons explicitly instead od the layout
- In this style, the above function might look like:

```
caseFunction x = case x of
 { 0 -> 1 ; 1 -> 5 ; 2 -> 2 ; _ -> 1 }
```

- It can also be written as

```
f x =
 case x of { 0 -> 1 ;
 1 -> 5 ; 2 -> 2
 ; _ -> 1 }
```

- However, this makes the code unreadable

108

Exercises

1. Use a combination of `fst` and `snd` to extract the character out of the tuple ((1,'a'),"foo").
2. Write a function `isConsonant` that when given a lower case character in the range a…z , determines whether it is a consonant
3. Write a function `power` x n, which raises x to the power of n . What is its type?

110